

Module 6: Asynchronous Programming in TypeScript

1. Introduction to Asynchronous Programming

Asynchronous programming allows non-blocking operations, improving application responsiveness.

2. Callbacks

Callbacks are functions passed as arguments and executed later.

typescript

```
function fetchData(callback: (data: string) => void) {  
  setTimeout(() => {  
    callback("Data received");  
  }, 2000);  
}
```

```
fetchData((data) => console.log(data));
```

3. Promises

A promise represents a future value.

typescript

```
let promise = new Promise((resolve, reject) => {  
  setTimeout(() => resolve("Promise resolved!"), 2000);  
});
```

```
promise.then((message) => console.log(message));
```

4. Async & Await

`async` and `await` simplify promise handling.

typescript

```
async function fetchData() {  
  return "Data fetched";  
}
```

```
async function main() {
```

```
let data = await fetchData();
console.log(data);
}

main();
```

5. Error Handling in Async/Await

Using `try...catch` to handle errors.

```
typescript

async function fetchData() {
  try {
    let response = await fetch("https://api.example.com/data");
    let data = await response.json();
    console.log(data);
  } catch (error) {
    console.log("Error fetching data", error);
  }
}

fetchData();
```

6. Using Fetch API

Fetching data from an API using `fetch()`.

```
typescript

fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then((response) => response.json())
  .then((data) => console.log(data))
  .catch((error) => console.log("Error:", error));
```

7. Parallel Execution with Promise.all()

`Promise.all()` executes multiple promises concurrently.

```
typescript

let promise1 = fetch("https://api.example.com/user");
```

```
let promise2 = fetch("https://api.example.com/posts");
```

```
Promise.all([promise1, promise2])  
.then((responses) => Promise.all(responses.map(res => res.json())))  
.then((data) => console.log(data));
```

Practice Questions:

1. Write a function that uses a callback to log "Hello, World!" after 2 seconds.
2. Create a promise that resolves with `"Task completed"` after 3 seconds.
3. Convert the following function into async/await:

typescript

```
function getUserData() {  
  return fetch("https://jsonplaceholder.typicode.com/users/1").then(res => res.json());  
}
```

4. Handle errors properly in the above function.
5. Use `Promise.all()` to fetch data from two different APIs simultaneously.

Practice Answers:

1. Callback function:

typescript

```
function delayedMessage(callback: () => void) {  
  setTimeout(() => callback(), 2000);  
}
```

```
delayedMessage(() => console.log("Hello, World!"));
```

2. Promise resolving after 3 seconds:

typescript

```
let taskPromise = new Promise((resolve) => {  
  setTimeout(() => resolve("Task completed"), 3000);  
});
```

```
taskPromise.then((message) => console.log(message));
```

3. Converted to async/await:

typescript

```
async function getUserData() {  
  let response = await fetch("https://jsonplaceholder.typicode.com/users/1");  
  return await response.json();  
}
```

4. Error handling in async function:

typescript

```
async function getUserData() {  
  try {  
    let response = await fetch("https://jsonplaceholder.typicode.com/users/1");  
    return await response.json();  
  } catch (error) {  
    console.log("Error fetching user data", error);  
  }  
}
```

5. Using `Promise.all`:

typescript

```
async function fetchMultipleAPIs() {  
  try {  
    let [user, posts] = await Promise.all([  
      fetch("https://jsonplaceholder.typicode.com/users/1").then(res => res.json()),  
      fetch("https://jsonplaceholder.typicode.com/posts/1").then(res => res.json())  
    ]);  
    console.log(user, posts);  
  } catch (error) {  
    console.log("Error fetching data", error);  
  }  
}
```

fetchMultipleAPIs();